

CO3-AT1 EXPERIMENT REPORT

Performance of Quick Sort Using Median-of-Three Pivot Selection

Course Code: CSA0606

Course Name: Design and Analysis of Algorithms (DAA)

Name: Sharmila R

Reg. No.: 192521213

1. State the Assessment Question

Problem Statement: Implement Quick Sort using three different pivot selection strategies: (1) First element as pivot, (2) Last element as pivot, and (3) Median-of-three pivot selection. Run the algorithms on large datasets with different input distributions and compare their execution times. Record and plot the results and analyze how median-of-three pivot selection reduces the possibility of worst-case performance.

Input Requirements: Large arrays of integers with different input distributions: Random data, Sorted data, Reverse-sorted data, and Nearly sorted data.

Expected Output: Sorted array, execution time for each pivot strategy, comparison table, and graphs showing execution time for different input distributions.

Constraints: Use the same input data for all three methods for fair comparison. Use sufficiently large datasets and record execution time in milliseconds.

Assessment Criteria: Correct Quick Sort implementation, correct pivot selection, accurate timing, proper tables/graphs, and correct interpretation.

2. Objective

To implement and evaluate Quick Sort using first-element, last-element, and median-of-three pivot selection strategies, and compare their execution performance on different input distributions.

3. Required Resources

Hardware Requirements: Computer/Laptop with a minimum of 4 GB RAM and a modern processor.

Software/Tools: Python 3.x, VS Code / Google Colab / Jupyter Notebook, and Matplotlib.

Programming Language: Python

IDE/Compiler: VS Code or Google Colab

Dataset/Input: Large integer datasets, for example 1,000; 5,000; 10,000; 50,000; and 100,000 elements.

4. Algorithm / Procedure

First Element Pivot: Select the first element as pivot. Partition the array around it. Place smaller elements on the left and larger elements on the right. Recursively apply Quick Sort to both partitions and record execution time.

Last Element Pivot: Select the last element as pivot. Partition the array, recursively sort both partitions, and record execution time.

Median-of-Three Pivot: Select the first, middle, and last elements. Find their median and

use it as the pivot. Partition the array, recursively sort both partitions, and record execution time.

Procedure: Start → Generate dataset → Select input distribution → Apply First Pivot Quick Sort → Record time → Apply Last Pivot Quick Sort → Record time → Apply Median-of-Three Quick Sort → Record time → Repeat for different input sizes → Store results → Plot graph → Compare and analyze → Stop.

5. Implementation

Source Code: The Python implementation uses a common partition function and three Quick Sort functions for first-element, last-element, and median-of-three pivot strategies. Execution time is measured using `time.perf_counter()`, and Matplotlib is used for plotting.

Important Functions: `partition()`, `quick_first()`, `quick_last()`, `median_of_three()`, `quick_median()`, and `measure()`.

Note: For the complete experiment, run the comparison separately for random, sorted, reverse-sorted, and nearly sorted data. For very large ordered datasets, first/last pivot strategies may reach Python's recursion limit because of their worst-case behavior.

6. Record Input and Output

Test Cases: Record the actual execution times obtained from your computer. Do not use made-up values.

7. Document the Results

Result Table: Prepare a table containing Input Distribution, Input Size, First Pivot Time (ms), Last Pivot Time (ms), and Median-of-Three Time (ms).

Graphs: Plot input size on the X-axis and execution time in milliseconds on the Y-axis. Use separate graphs for each input distribution when appropriate.

8. Analyze the Results

Expected Observation: For random data, all strategies generally perform reasonably well. For sorted and reverse-sorted data, first/last pivot selection can create highly unbalanced partitions. Median-of-three generally chooses a pivot closer to the middle. Nearly sorted data can also expose weaknesses of simple first/last pivot selection.

Why Median-of-Three Is Effective: Median-of-three chooses the median among the first, middle, and last elements. This reduces the chance of selecting an extreme value and usually produces more balanced partitions.

Time Complexity: Best Case: $O(n \log n)$. Average Case: $O(n \log n)$. Worst Case: $O(n^2)$. Median-of-three reduces the likelihood of poor pivot choices but does not eliminate the theoretical $O(n^2)$ worst case.

Space Complexity: Average recursion space: $O(\log n)$. Worst-case recursion space: $O(n)$.

Advantages: Simple pivot improvement; generally more balanced partitions; useful for partially sorted data; reduces the chance of worst-case behavior; suitable for large datasets.

Limitations: Requires extra comparisons to select the median; does not guarantee $O(n \log n)$; specially constructed inputs can still cause poor performance; timing varies with system conditions.

Comparison: First Element: simple but poor for ordered data. Last Element: simple but poor for ordered data. Median-of-Three: more robust in practice, while still having $O(n^2)$ worst-case complexity.

9. Conclusion

The experiment successfully implemented Quick Sort using first-element, last-element, and median-of-three pivot selection strategies. Their execution times were measured and compared using different input distributions and dataset sizes. First-element and last-element strategies can perform poorly on sorted or reverse-sorted data because they may repeatedly produce highly unbalanced partitions. Median-of-three generally provides more balanced partitions and reduces the likelihood of worst-case behavior. Therefore, median-of-three is an effective pivot selection technique for improving the practical performance of Quick Sort on large and partially ordered datasets, although it does not completely eliminate the theoretical $O(n^2)$ worst case.

Suggested Result Table

Input Distribution	Input Size	First Pivot (ms)	Last Pivot (ms)	Median-of-Three (ms)
Random	1,000	Actual value	Actual value	Actual value
Random	5,000	Actual value	Actual value	Actual value
Sorted	1,000	Actual value	Actual value	Actual value
Reverse Sorted	1,000	Actual value	Actual value	Actual value
Nearly Sorted	5,000	Actual value	Actual value	Actual value